rogii-wellbore-geology-prediction

1st Place Solution

Rank	1
Team	Ruby
Author	Ruby
Topic ID	733220
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733220
Retrieved with Kaggle CLI on 2026-08-06.

Thanks to Kaggle and the competition hosts for organizing such an interesting competition. I was very lucky to survive the leaderboard shake-up and win my first 1st place gold medal.

I'd also like to thank everyone who shared ideas publicly, especially @tuckerarrants , who motivated me to explore neural networks early in the competition, and whoever originally shared the particle filter baseline (unfortunately I couldn't trace the original author).

Most of the code was written with Codex using GPT-5.5/5.6. The repository contains many implementation details, so this write-up only covers the ideas that I believe contributed most of the performance gains. For the full implementation, I recommend giving the code to an AI coding agent and asking it to explain the details. The repository also contains many inactive experimental features, which can safely be ignored.

Task Formulation and Loss

I formulate the problem as a **2D alignment task** and use **cross-entropy loss** as the main objective.

The alignment grid is:

- **Horizontal well:** 345 positions (downsampled by 32), consisting of 1,024 visible plus 10,000 target region.
- **Typewell:** 400 positions covering ±100 ft around the last visible TVT with 0.5-ft resolution.

Throughout this write-up, all TVT values are **relative to the last visible TVT**, which serves as the anchor.

The training target is an exponentially smoothed probability distribution centered around the ground-truth alignment and normalized along the typewell dimension.

Additional losses:

- **Huber loss** on the expected TVT path, computed from the predicted probability map and grid TVT values.
- **GR penalty loss:** `mean(probability × grid_GR_gap)`, which provides a small additional improvement.

Model

I use a standard **2D U-Net** with a ConvNeXt backbone:

- `timm/convnext_small.in12k_ft_in1k_384`
- Standard residual blocks in the decoder.

Replacing **LayerNorm** in ConvNeXt with **BatchNorm** consistently works better, although BF16 training is required to avoid NaN losses.

For downsampling and upsampling, simple **average pooling + interpolation** performs better than learnable alternatives.

Features

Typewell

- GR
- GR is NaN
- TVT

Typewell GR is calibrated using the visible region by aggregating `(TVT, GR)` into TVT bins, interpolating back to the typewell axis, and blending with the original GR values.

Horizontal well

For each downsampled bin:

- GR mean
- GR nan rate
- GR standard deviation
- GR slope
- Last − first GR
- Quadratic fitting coefficients
- Quadratic fitting residual RMSE
- Visible-region TVT mean

Typewell × Horizontal interactions

- `|typewell_GR - horizontal_GR|`
- `|typewell_TVT - visible_TVT|`

Coordinate based

- `z_diff`

Particle-filter features

- 2D particle probability heatmap
- `|typewell_TVT - PF_TVT|`

XY-neighbor features

- Predicted TVT difference
- Predicted TVT
- `|typewell_TVT - predicted_TVT|`

When particle-filter channels are present (which already contain accumulated TVT information), using accumulated TVT performs better than using only TVT differences.

Particle Filter

This component was largely developed with the help of AI coding agents since I was not very familiar with particle filters. The standalone local CV is approximately **7.4 RMSE**.

Compared with the public baseline, I believe the major improvements come from:

- Allowing low-probability large jumps
- Calibrated typewell GR
- FFBSi smoothing
- Blending diverse cfg profiles
- Updating particles in bins (64 samples) instead of raw resolution

There are also many smaller implementation details that can be found in the released code.

XY Neighbor Information

Assume the local geological surface satisfies

> **S = TVT + Z + C**

and S is locally linear, then

> **ΔTVT = aΔX + bΔY − ΔZ**

where `(a, b)` are estimated by weighted least squares using (x,y) neighbors.

The implementation includes anisotropic distance, singular-case handling, regularization, and several other engineering details. However, these contribute only marginal improvements over the simple formulation above, so I won't discuss them further.

The standalone local CV is approximately **11.4 RMSE**.

Adding XY-neighbor information consistently **improved local CV** but **hurt the public leaderboard**, so I performed several analyses.

1. XY-neighbor prediction alone scores **12.9 LB** which lies in high prob region for 50 wells sample.
2. I replaced XY based predictions with GR + `z_diff` predictions whenever neighborhood statistics exceeded the 95th percentile of the training distribution.

The neighborhood statistics include:

- Mean neighbor distance
- 10th percentile neighbor distance
- Prefix/visible neighbor weight ratio
- Distance(query, neighbor center) / average distance(neighbor, neighbor center)
- Average abs cosine corr((x_diff,y_diff),(x_nbr_diff,y_nbr_diff))

None of these statistics explained the leaderboard degradation, and I believe they sufficiently describe neighborhood quality.

Therefore, I attribute the discrepancy to inconsistent labels and chose to trust the local CV, which consistently improves by about **0.3 RMSE**.

These same neighborhood quality statistics are also used later in the ensemble.

Data Augmentation

The two most important augmentations are **Z-shift** and **GR transform**.

Z-shift

Randomly sample a TVT path while keeping **TVT + Z** unchanged.

The sampled path is generated using block bootstrap on real TVT differences.

GR is regenerated by matching the typewell using:

- TVT + TVT noise

where the TVT noise consists of smoothed white noise.

Rare fault jumps are also simulated, introducing offsets in **TVT + Z**.

GR transform

Apply

> `GR' = a × GR + b`

to the typewell GR, forcing the model to rely more on shape than absolute values.

Other augmentations

- Reverse path (reverse traversal while keeping part of the beginning visible)
- MD stretching
- 2D channel masking
- GR noise shift (shift residual `(GR - matched_typewell_GR)` within each well)
- Tail cropping
- Sequential masking along the typewell axis
- Typewell GR jitter (consistent transformation of GR channels)
- PF rotate/shift (corrupt PF channels to reduce shortcut learning)

Ensemble

For wells without reliable XY-neighbor information (approximately **10%**), I use models without XY-neighbor channels and include the `z_diff` channel instead.

Below, "default" refers to models using GR, TVT, and `z_diff` channels without particle-filter or XY-neighbor features. Models also differ slightly in augmentation and training configurations.

Model	Features	Weight (general / no XY)	CV (3 seeds × 5 folds avg)	Public LB	Private LB
Weighted ensemble	—	—	**4.627**	**5.980**	**5.639**
0719_V1	Default	0.07 / 0.40	5.09	5.648	6.130
0729_V3	+ Particle filter	0.00 / 0.20	5.53	6.202	6.768
0801_V1	Default	0.07 / 0.40	5.16	5.723	5.884
0724_V1	+ XY-neighbor channels	0.28 / 0.00	4.86	6.095	5.831
0801_V2	+ XY-neighbor channels	0.28 / 0.00	4.80	6.166	5.937
0803_V2	+ XY-neighbor channels + Particle-filter channels	0.28 / 0.00	5.00	6.185	5.778

Code

The inference notebook is available here:

https://www.kaggle.com/code/w5833946/submit-reproduce

It also explains how to reproduce the full training pipeline, each 3 seeds X 5 folds training takes around 8 hours on single 4080s.